

SECTION 1: WHAT TESTING IS AND IS NOT

Software testing is the process of evaluating a system or its components with the intent of finding whether it satisfies the specified requirements. Testing is an investigation carried out to give stakeholders information about the quality of the product under test.

A central principle is that testing can show the presence of defects but cannot prove their absence. Running a test that passes tells us only that the system behaved correctly for that particular input on that particular run. Exhaustive testing, meaning testing every possible combination of inputs and preconditions, is not feasible for any non-trivial system, so testing is always a sampling activity guided by risk.

Verification asks whether the product was built correctly, that is, whether it conforms to its specification. Validation asks whether the correct product was built, that is, whether it meets the actual needs of the user. A system can pass verification and still fail validation if the specification itself was wrong.

A defect, also called a bug or fault, is a flaw in a component or system that can cause it to fail to perform its required function. A failure is the observable event in which the system does not perform as required. An error is the human action that produced the defect. The chain therefore runs from error to defect to failure.

The cost of defect correction rises sharply the later a defect is found in the development lifecycle. A defect found during requirements analysis may cost very little to correct, while the same defect found in production may cost orders of magnitude more because of rework, redeployment and damage to user confidence. This economic argument is the primary justification for early and continuous testing.

SECTION 2: THE SEVEN TESTING PRINCIPLES

The first principle is that testing shows the presence of defects, not their absence.

The second principle is that exhaustive testing is impossible, so effort must be focused using risk analysis and priorities.

The third principle is early testing. Testing activities should begin as early as possible in the development lifecycle, including the review of requirements and design documents before any code exists.

The fourth principle is defect clustering. A small number of modules usually contains the majority of the defects discovered before release. This observation is a practical application of the Pareto principle and it justifies concentrating effort on historically defect-prone areas.

The fifth principle is the pesticide paradox. If the same tests are repeated over and over, they eventually stop finding new defects. Test cases must be reviewed and revised regularly, and new tests written, in order to keep finding new problems.

The sixth principle is that testing is context dependent. A safety-critical medical device is tested very differently from a marketing website, because the consequences of failure differ.

The seventh principle is the absence of errors fallacy. Finding and fixing a large number of defects is of no value if the system that results is unusable or does not meet the users' needs.

SECTION 3: LEVELS OF TESTING

Unit testing examines individual components in isolation, usually functions, methods or classes. It is normally performed by the developer who wrote the code, and it relies on stubs and mocks to stand in for collaborators that are not under test. Unit tests are fast, numerous and cheap to run, which makes them suitable for execution on every change.

Integration testing examines the interfaces and interactions between components that have already been unit tested. It finds defects in data passed between modules, in interface contracts, and in assumptions one component makes about another. Two common strategies are top-down integration, which uses stubs for lower components, and bottom-up integration, which uses drivers for higher components. Big-bang integration, in which all components are combined at once, is generally discouraged because it makes fault localisation difficult.

System testing examines the complete, integrated system against the specified requirements. It is performed in an environment as close to production as practical, and it covers both functional and non-functional requirements.

Acceptance testing determines whether the system is acceptable to the customer or end user. User acceptance testing is performed by representatives of the intended users. Alpha testing is conducted at the developer's site by people who are not the development team, while beta testing is conducted at the customer's site by real users under real conditions.

SECTION 4: BLACK BOX TEST DESIGN TECHNIQUES

Black box techniques derive test cases from the specification without reference to the internal structure of the code.

Equivalence partitioning divides the input domain into partitions within which all values are expected to be treated the same way by the system. One value is then chosen to represent each partition, on the assumption that if that value is handled correctly the others will be too. Partitions exist for both valid and invalid inputs, and both must be tested.

Boundary value analysis complements equivalence partitioning by testing at the edges of each partition, because defects cluster at boundaries. For a field accepting values from one to ninety-nine, the boundary values to test are zero, one, two, ninety-eight, ninety-nine and one hundred. The technique reflects the frequency of off-by-one errors in comparison operators.

Decision table testing captures combinations of conditions and the actions that should result from each combination. It is well suited to business rules where several conditions interact, and it exposes combinations the specification failed to address.

State transition testing models the system as a set of states with permitted transitions between them, driven by events. It is appropriate where the system's response to an input depends on its history, such as a login process that locks an account after three failed attempts. Tests cover valid transitions, and also invalid transitions that should be rejected.

Use case testing derives tests from the interactions described in use cases, exercising the main success scenario together with the documented alternative and exception flows.

SECTION 5: WHITE BOX TEST DESIGN TECHNIQUES

White box techniques, also called structural techniques, derive tests from the internal structure of the code.

Statement coverage measures the percentage of executable statements exercised by a test suite. It is the weakest of the common coverage measures, because a suite can reach full statement coverage without ever taking the false branch of a condition that has no else clause.

Decision coverage, also called branch coverage, measures the percentage of decision outcomes exercised. Full decision coverage implies full statement coverage, but the converse does not hold.

Condition coverage measures whether each individual boolean sub-expression within a decision has been evaluated both true and false. Modified condition decision coverage additionally requires demonstrating that each condition independently affects the decision outcome, and it is mandated for avionics software at the highest criticality levels.

Path coverage requires exercising every possible route through the code. The number of paths grows combinatorially with the number of decisions and becomes impractical for real programs, particularly where loops are involved.

Coverage is a useful indicator of what has not been tested, but a poor measure of test quality. Code can be executed by a test that asserts nothing, which registers as covered while verifying nothing at all.

SECTION 6: NON-FUNCTIONAL TESTING

Performance testing determines how a system behaves in terms of responsiveness and stability under a particular workload. Load testing applies an expected level of demand. Stress testing applies demand beyond the expected maximum in order to find the breaking point and to observe how the system fails. Soak testing, also called endurance testing, applies a sustained load over an extended period to reveal problems such as memory leaks that only appear over time. Spike testing applies a sudden sharp increase in load.

Usability testing evaluates how easily intended users can accomplish their goals, and is normally conducted by observing representative users attempting representative tasks.

Security testing seeks to establish that the system protects data and maintains functionality as intended, covering areas such as authentication, authorisation, input validation, session management and the secure storage of credentials.

Compatibility testing establishes that the system operates correctly across the intended range of browsers, devices, operating systems and screen sizes.

Accessibility testing establishes that the system can be used by people with disabilities, and is commonly assessed against the Web Content Accessibility Guidelines.

Reliability testing measures the ability of the system to perform its